Бинарные деревья

Бинарное дерево - структура данных, которая обязательно содержит корень и указатели на двух потомков (поддеревья), которые также являются бинарными деревьями

Бинарное дерево поиска (БДП) - бинарное дерево, в левом поддереве которого все значения меньше узла, в правом - больше

Поиск в БДП:

1. Сравнить число x с числом в узле.
   - Если x меньше, то пойти в левого потомка. Вернуться к п.1.
   - Если x больше, то пойти в правого потомка. Вернуться к п.1.
   - Если x равно, то число найдено в дереве.
   - Если узел неопределён (`nullptr`), то числа в дереве нет.

Сложность: В лучшем случае $O(\log n)$, в худшем случае $O(n)$ - связанный список.

Вставка в БДП:

1. По правилам аналогичным поиску спустится к листу дерева.
2. Если вставляемое число меньше или равно значению узла, то записывать его в левого потомка.
3. Если вставляемое число больше больше значения узла, то записать его в правого потомка.

Сложность: $O(h)$, где h - максимальная высота, которая может быть равна как $\log n$, так и просто n.

Удаление в БДП:

1. Воспользоваться поиском, чтобы найти число.
2. Если такого числа нет в дереве, то удалять нечего.
3. Если есть, то необходимо рассмотреть следующие случаи:
   1. Если у узла нет потомков, то для его предка просто убираем указатель на узел, сам узел удаляем из памяти.
   2. Если у узла один потомок, то для его предка указатель делаем на этого потомка, сам узел удаляем из память.
   3. Если у узла два потомка, то находим минимальное значение в правом поддереве, перезаписываем это значение. Найденный узел, который перезаписали, пытаемся удалить согласно п.3.1 или п.3.2.

Сложность: $O(h)$, где h - максимальная высота дерева.

Повороты в бинарных деревьях

Левый поворот

Пусть X - узел ($x \in X$), Y - правый потомок X ($y \in Y \quad y > x$), α - некоторое левое поддерево X ($\forall a \in \alpha \quad a < x$), β - некоторое левое поддерево Y ($\forall b \in \beta \quad b < y$), γ - некоторое правое поддерево Y ($\forall z \in \gamma \quad z > y$).

Левым поворотом называется перестановка узлов, когда Y становится главным узлом, его правым потомком становится X, левым остаётся γ, прчём α остаётся левым потомком X, а β становится правым потомком X.

Сложность: $O(1)$, т.к. все перестановки делаются путём переставления указателей за константу без выделения дополнительной памяти.

Правый поворот

Пусть X - узел ($x \in X$), Y - левый потомок X ($y \in Y \quad y < x$), γ - некоторое правое поддерево X ($\forall z \in \gamma \quad z > x$), α - некоторое левое поддерево Y ($\forall a \in \alpha \quad a < y$), β - некоторое правое поддерево Y ($\forall b \in \beta \quad b > y$).

Правым поворотом называется перестановка узлов, когда Y становится главным узлом, его левым потомком становится X, правым остаётся α, прчём γ остаётся парвым потомком X, а β становится левым потомком X.

Сложность: $O(1)$, т.к. все перестановки делаются путём переставления указателей за константу без выделения дополнительной памяти.

AVL дерево

AVL деревом (АВЛ деревом) называется бинарное дерево поиска, удовляетворяющие условию, что для любого узла $X \quad |h(X->left) - h(X->right)| \leq 1$, где $h(x)$ - функция высота для каждого узла, т.е. разница высот между правым и левым поддеревьями узла X по модулю не превышает 1.

Поиск в AVL

Аналогично поиску в БДП.

Сложность: $O(h)$, где h - максимальная высота дерева

Балансировка AVL дерева

Пусть в узле X вместе с числом x хранится переменная $balance = (X->left) - (X->right)$. Тогда для сохранения свойства AVL-дерева $balance \in [-1, 0, 1]$. При вставка или удалении какого-либо узла может произойти такое, что $balance \in [-2, -1, 0, 1, 2]$. В таком случае применяются правила балансировки:

1. Если после вставки или удаления в узле X значение $|balance| = 0$, то можно закончить процесс балансировки.
2. Если после вставки или удаления в узле X значение $|balance| = 1$, то необходимо рекурсивно вверх продолжить проверку.
3. Если после вставки или удаления в узле X значение $|balance| = 2$, то:
    1. Если $X \to balance = -2$ и $X \to right \to balance \leq 0$, то делаем малый левый поворот для X и $X \to right$.
    2. Если $X \to balance = -2$ и $X \to right \to balance > 0$, то делаем большой левый поворот: сначала правый для $X \to right$ и $X \to right \to left$, потом левый для X и $X \to right$.
    3. Если $X \to balance = 2$ и $X \to left \to balance \geq 0$, то делаем малый правый поворот для X и $X \to left$.
    4. Если $X \to balance = 2$ и $X \to left \to balance < 0$, то делаем большой правый поворот: сначала левый для $X \to left$ и $X \to left \to right$, потом правый для X и $X \to left$.

Сложность: $O(1)$.

Вставка в AVL дерево

Алгоритм:

1. Запустить поиск для нахождения `nullptr`, куда можно вставить значение.
2. Если такое место было найдено, то установить его $balance = 0$.
3. Запустить рекурсивную балансировку для всех его предков, пока после изменения баланса в одном из предков не станет 0 или не будет достигнут корень.

Сложность: $O(h)$, где h - высота дерева.
Доказательство: Поиск $O(h)$, балансировка в худшем случае до корня $O(h)$. Итого: $O(h + h) = O(2h) \approx O(h)$.

Удаление из AVL дерева

Алгоритм:

1. Запустить поиск для нахождения узла.
2. Если узел не найден, то ничего не делаем.

3. Если узел найден, то рассмотрим следующие случаи:
    1. Удаляемый узел не имеет детей. Удаляем его, для предка запускаем рекурсивную проверку баланса.
    2. Удаляемый узел имеет одного потомка. Даём предку нашего потомка, удаляем узел, запускаем рекурсивную проверку баланса начиная с потомка.
    3. Удаляемый узел имеет двух потомков. В правом потомке находим узел с минимальным значением. Перезаписываем это в узел, который хотели удалить. Согласно п.3.1 или п.3.2 пытаемся удалить узел, значение которого было перезаписано.

Сложность: $O(h)$, где h - высота дерева.
Доказательство: Поиск $O(h)$, балансировка в худшем случае $O(h)$, поиск минимального узла в потомке в худшем случае $O(h)$. Итого: $O(h + h + h) = O(3h) \approx O(h)$.

Теорема о высоте AVL дерева

Теорема: Для AVL дерва с n узлами максимальная высота $h \approx \log n$.
Доказательство: Чтобы построить AVL-дерево высоты h с минимальным числом узлов, у его корня одно поддерево должно иметь высоту $h - 1$ (чтобы обеспечить общую высоту h), а второе поддерево должно иметь высоту $h - 2$ (это минимально возможная высота второго поддерева, при которой не нарушается свойство AVL, то есть разница высот равна ровно 1).
Отсюда получаем рекуррентное соотношение для количества узлов:

$$N(h) = N(h-1) + N(h-2) + 1,$$

где `+1` это сам корень.
Очевидно, что

$$N(h-1) > N(h-2) \quad \Rightarrow \quad N(h) > N(h-2) + N(h-2) + 1 > 2 \cdot N(h-2).$$

Разворачивая индукцию получаем:

$$N(h) > 2 \cdot N(h-2) > 2^2 \cdot N(h-4) > \ldots > 2^{h/2}.$$

Логарифмируем обе части: $\log N(h) > \frac{h}{2} \Rightarrow h < 2 \log N(h)$.

Т.к. $n > N(h)$, получаем:

$$h < 2 \log n \Rightarrow h = O(\log n).$$